13. Liste, cozi, stive şi arbori

Un program constă din două componente: *algoritmi* şi *structuri de date*. Un program de calitate este o combinație reuşită a ambelor componente. Alegerea şi modul de implementare ale unei structuri de date are o importanță egală cu aceea a rutinelor necesare la manevrarea acesteia. Modul de organizare şi de accesare a informației este de obicei determinat de natura problemei de programare. Din acest motiv, este important să dispuneți în orice moment de metoda corespunzătoare de stocare şi găsire a datelor.
Modul de legare a unui tip de date de reprezentarea sa în calculator este invers corelată cu gradul său de abstractizare. Cu alte cuvinte, pe măsură ce tipurile de date sunt mai abstracte, modul în care le concepe mental programatorul are o legătură din ce în ce mai redusă cu modul în care sunt acestea reprezentate în memorie. Tipurile simple de date, cum ar fi `char` şi `int`, sunt strâns legate de modul de reprezentare. Tablourile simple, care constituie ansambluri organizate de tipuri de date de bază, nu sunt atât de legate ca tipurile elementare, dar modul de existență al acestora în memorie este asemănător cu modul de vizualizare.
Pe măsură ce complexitatea unui tip de date creşte, acestea devin conceptual mai puțin similare cu echivalentele lor din memorie. De exemplu, valorile în virgulă mobilă sunt mai abstracte decât întregii. Structura, care reprezintă un conglomerat de tipuri de date, este încă şi mai complexă. Nivelul final de abstractizare transcende simplul aspect fizic al datelor, prin adăugarea mecanismului de accesare a datelor, adică de stocare şi recuperare a acestora. În esență, datele fizice sunt legate cu un *motor de date (data engine),* care controlează modul de accesare a acestora de către un program. Există patru mari categorii de motoare de date:

■ Lista înlănțuită ■ Coada ■ Stiva ■ Arborele binar

Fiecare dintre aceste metode asigură o soluție pentru o anumită clasă de probleme. În esență, metodele de mai sus reprezintă dispozitive care efectuează o anumită operație de stocare şi recuperare asupra informațiilor şi cererilor primite. Toate aceste metode stochează un articol şi recuperează un articol, unde prin articol se înțelege o unitate informațională.

13.1 Liste

Lista este o mulțime *dinamică* cu un număr variabil de elemente.

La început lista este o mulțime vidă. În procesul execuției programului se pot adăuga elemente noi listei şi totodată pot fi eliminate diferite elemente din listă de care nu mai este nevoie. Deci, elementele unei liste sunt structuri de acelaşi tip. Se obişnuieşte să se ordoneze elementele unei liste folosind pointeri care intră în compunerea elementelor listei. Datorită acestor pointeri, elementele listei devin *structuri recursive*. Listele organizate în acest fel se numesc *liste înlănțuite.*

Listă înlănțuită este o mulțime dinamică de structuri recursive de acelaşi tip, pentru care sunt definite una sau mai multe relații de ordine cu ajutorul unor pointeri din compunerea structurilor respective. Elementele unei liste, de obicei, se numesc ***noduri***.

Dacă între nodurile unei liste există o singură relație de ordine, atunci lista se numeşte ***simplu înlănțuită***. *În* mod analog, lista este ***dublu înlănțuită*** dacă între nodurile ei sunt definite două relații de ordine.
În general, vom spune că o listă este *n-înlănțuită* dacă între nodurile ei sunt definite *n* relații de ordine.
În legătură cu listele înlănțuite se au în vedere unele operații de interes general:

a. crearea unei liste înlănțuite;
b. accesul la un nod oarecare al unei liste înlănțuite;
c. inserarea unui nod într-o listă înlănțuită;
d. ştergerea unui nod dintr-o listă înlănțuită;
e. ştergerea unei liste înlănțuite.

13.1.1 Liste simplu înlănțuite

Aşa cum s-a arătat mai sus, între nodurile unei liste simplu înlănțuite (*simple-linked*) este definită o singură relație de ordonare. De obicei această relație este cea de succesor, adică fiecare nod conține un pointer a cărui valoare reprezintă adresa nodului *următor* din listă. În mod analog se poate defini relația de *precedent. În* cele ce urmează ne vom mărgini numai la liste simplu înlănțuite pentru care nodurile satisfac relația de succesor. Într-o astfel de listă există totdeauna un nod şi numai unul care nu mai are următor (succesor), precum şi un nod care nu este următorul (succesorul) nici unui alt nod. Aceste noduri formează *capetele listei* simplu înlănțuite.

O *listă înlănțuită (linked list)* poate fi accesată într-o manieră aleatorie, deoarece fiecare entitate informațională este dotată cu o legătură cu următorul articol de date din listă, şi din acest motiv lista înlănțuită necesită o structură de date complexă; în plus, o operație de recuperare asupra unei liste înlănțuite nu elimină şi nu distruge un articol din listă. Pentru a realiza aceasta, este nevoie de o operație specifică de ştergere.

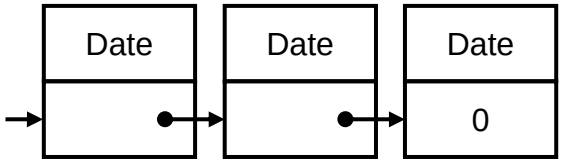


Figura 13.1 Listă simplu înlănțuită

Fiecare articol de date constă dintr-o structură care conține câmpul informațional şi un pointer de legătură. Conceptual, o listă simplu înlănțuită are aspectul prezentat în **Error! Reference source not found.**.
În esență, există două moduri de a alcătui o listă înlănțuită simplă. Primul rezidă în inserarea fiecărui articol nou la sfârşitul listei. Celălalt constă în adăugarea articolelor noi în locuri bine determinate, de exemplu în ordine crescătoare. Modul de alcătuire a listei va determina codarea funcției de stocare. Să începem cu situația mai simplă a creării unei liste înlănțuite folosind adăugarea articolelor la sfârşitul listei.
În general, articolele stocate într-o listă înlănțuită constau din structuri, deoarece fiecare articol trebuie să conțină atât legătura cu articolul următor de pe listă, cât şi datele în sine. Ca atare, va fi necesară definirea unei structuri care va fi folosită în exemplele următoare. Deoarece listele de adrese sunt adesea stocate într-o listă înlănțuită, o structură tip adresă este oportună. Structura de date pentru fiecare element din lista de adrese este definită mai jos:

```
struct adresa {
char nume[40];
char strada[40];
char oras[20] ;
char stat[3];
char cod_postal [11];
struct adresa *urm; }  info;
```

Funcția **store_ls(),** prezentată în cele ce urmează, construieşte o listă simplu înlănțuită prin adăugarea fiecărui element nou la sfârşitul listei. Este necesar transferul unui pointer către o structură de tip **adresa,** precum şi al unui pointer către ultimul element din listă. Dacă lista este goală, atunci pointerul ultimului element din listă trebuie să fie nul.

```
void store_ls(struct adresa *i, struct adresa **ultim)
{
 if(!*ultim) *ultim = i; /* primul articol din lista */
 else (*ultim)->urm = i;
 i->urm = NULL;
 *ultim = i;
}
```

Deşi lista creată prin intermediul funcției **store_ls()** poate fi sortată printr-o operație separată, este mai simplă sortarea listei în timpul construirii acesteia, prin inserarea fiecărui element nou în secvența corespunzătoare a lanțului. De asemenea, dacă lista este deja sortată, este avantajoasă menținerea sortării prin inserarea articolelor noi în locația corespunzătoare fiecăruia. Această operație se realizează printr-o parcurgere secvențială a listei până la găsirea locației corespunzătoare, inserarea noii adrese în acel loc şi rearanjarea legăturilor.

La inserarea unui articol într-o listă simplu înlănțuită pot surveni trei situații posibile: articolul poate deveni primul pe listă, poate fi plasat între alte două articole sau poate deveni ultimul articol. Diagramele din **Error! Reference source not found.** indică modificarea legăturilor pentru fiecare caz în parte.

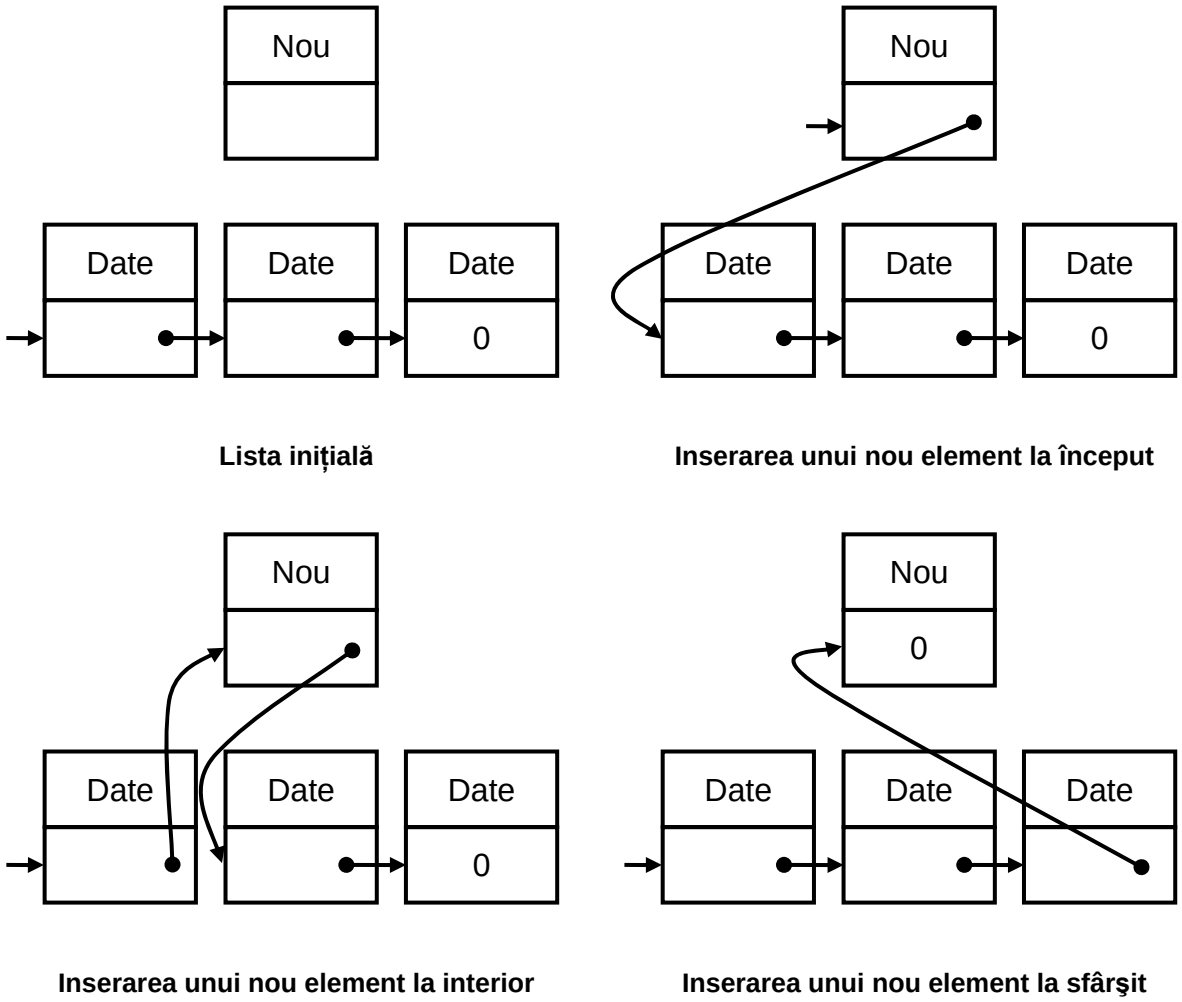


Figura 13.2 Inserarea unui nou element într-o listă simplu înlănțuită

Rețineți că dacă schimbați primul articol dintr-o listă, trebuie să actualizați punctul de intrare în listă peste tot în program. Pentru a evita această sarcină, puteți folosi ca prim articol o *santinelă (sentinel)*. În acest caz, selectați o valoare de intrare. Această metodă are dezavantajul unei locații suplimentare pentru memorarea valorii-santinelă, dar acesta nu este un factor de prea mare importanță.

Funcția prezentată în cele ce urmează, `store_ls_sort()`, va insera structuri de tip **adresa** în lista de adrese într-o ordine ascendentă bazată pe câmpul nume. Este necesar transferul unui pointer către pointerul primului şi ultimului element din listă, o dată cu un pointer către informația care va fi stocată. Deoarece primul sau ultimul element din listă se poate modifica, funcția `store_ls_sort()` actualizează automat pointerii către începutul, respectiv sfârşitul listei, dacă aceştia se modifică. La primul apel din program către funcția `store_ls_sort()`, *prim* şi *ultim* trebuie să fie pointeri nuli.

```
/* Stocare in ordine ascendenta */
void store_ls_sort (struct adresa *i, /*noul element de stocat */
 struct adresa **prim, /* inceputul listei */
 struct adresa **ultim) /* sfarsitul listei */
{
 struct adresa *vechi, *p;
 p = *prim;
 if(!*ultim) { /* primul element din lista */
 i->urm = NULL;
 *ultim = i;
 *prim = i;
```

```
return;
}
vechi = NULL;
while(p) {
 if(strcmp(p->nume, i->nume) <0) { vechi = p; p = p->urm; }
 else {
  if(vechi) { /* intra la mijloc */
    vechi->urm = i; i->urm = p; return;
  }
 i->urm = p; /* noul prim element */
 *prim = i; return;}
}
(*ultim)->urm = i; /*introduce la sfârşit */
i->urm = NULL; *ultim = i; }
```

O listă înlănțuită dispune rareori de o anumită funcție dedicată procesului de citire a datelor, adică de returnare a unui articol după altul în ordinea din listă. De obicei, această funcție este atât de scurtă, încât este pur şi simplu plasată în interiorul unei alte rutine, cum ar fi o funcție de căutare, ştergere sau afişare. De exemplu, rutina de mai jos afişează toate numele dintr-o listă de adrese:

```
void display(struct adresa *prim)
{
 while(prim) {printf("%s\n", prim->nume); prim = prim->urm; }
}
```

La apelarea funcției `display()`, prim trebuie să fie un pointer către prima structură din listă.
Citirea articolelor dintr-o listă se rezumă la urmărirea lanțului. O rutină de căutare bazată pe câmpul *nume* ar putea fi scrisă după cum urmează:

```
struct adresa *search(struct adresa *prim, char *n)
{
 while(prim) {
  if(!strcmp(n, prim->nume)) return prim;
  prim = prim->urm; }
 return NULL; /* nici o coincidenta */
}
```

Deoarece funcția `search()` returnează un pointer către articolul din listă care corespunde numelui căutat, trebuie declarată ca returnând un pointer de structură de tip adresa. Dacă nu există nici o coincidență, se va returna un pointer nul.
Ştergerea unui articol dintr-o listă înlănțuită simplă este o operație directă. Ca şi în cazul inserării unui articol, există trei situații: ştergerea primului articol, ştergerea unui articol din interiorul listei şi ştergerea ultimului articol. În Fig 13.3 sunt prezentate toate aceste operații.

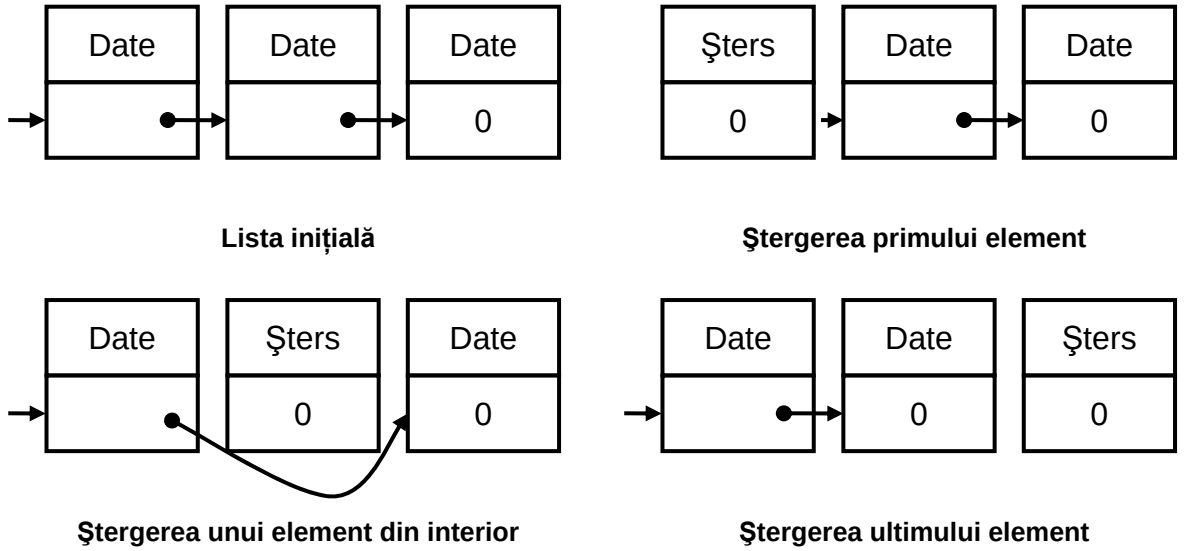


Figura 13.3 Ştergerea unui element dintr-o listă simplu înlănțuită

Funcția următoare şterge un articol dat dintr-o listă de structuri de tip adresa:

```
void sterg_ls (
struct adresa *p, /* articolul precedent */
 struct adresa *a, /* articolul care trebuie şters */
 struct adresa **prim, /* inceputul listei */
 struct adresa **ultim, /* sfârşitul listei */
{
 if(p) p->urm = a->urm;
 else *prim = a->urm;
 if(i==*ultim &&p) *ultim = p;
}
```

Funcția `sterg_ls()` trebuie să primească pointerii articolului şters, al articolului precedent din lanț, precum şi ai primului şi ultimului articol din listă. Funcția actualizează automat pointerii prim şi ultim, în cazul în care articolul indicat de vreunul dintre aceştia este şters.

Listele simplu înlănțuite au o **deficiență majoră** care le interzice utilizarea pe scară largă, şi anume imposibilitatea de a fi citite în ordine inversă. Din acest motiv, cele mai des folosite sunt listele dublu înlănțuite.

13.1.2 Liste dublu înlănțuite

Listele dublu înlănțuite sunt alcătuite din date la care se adaugă atât legături cu următorul articol, cât şi cu precedentul. Fig 13.4 indică modul de dispunere al acestor legături.

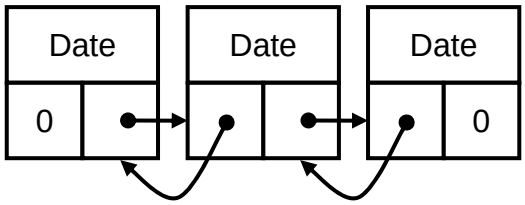


Figura 13.4 Listă dublu înlănțuită

Existența a două legături în locul uneia singure prezintă mai multe avantaje, din care poate cel mai important rezidă în faptul că lista poate fi citită în ambele direcții. Aceasta simplifică gestiunea datelor din listă şi facilitează introducerea şi ştergerea acestora, permițând utilizatorului o parcurgere a listei în orice direcție. Un alt avantaj se manifestă numai într-un anumit caz de eroare. Din moment ce întreaga listă poate fi consultată folosind fie legăturile dintr-un sens, fie dintr-altul, dacă una dintre legături suferă un defect, lista poate fi reconstruită folosind cealaltă legătură.

Există trei modalități de inserare a unui nou element într-o listă dublu înlănțuită: inserarea unui nou prim element, introducerea unui element între alte două existente sau inserarea unui nou element ultim. Aceste operații sunt ilustrate în Fug 13.5.

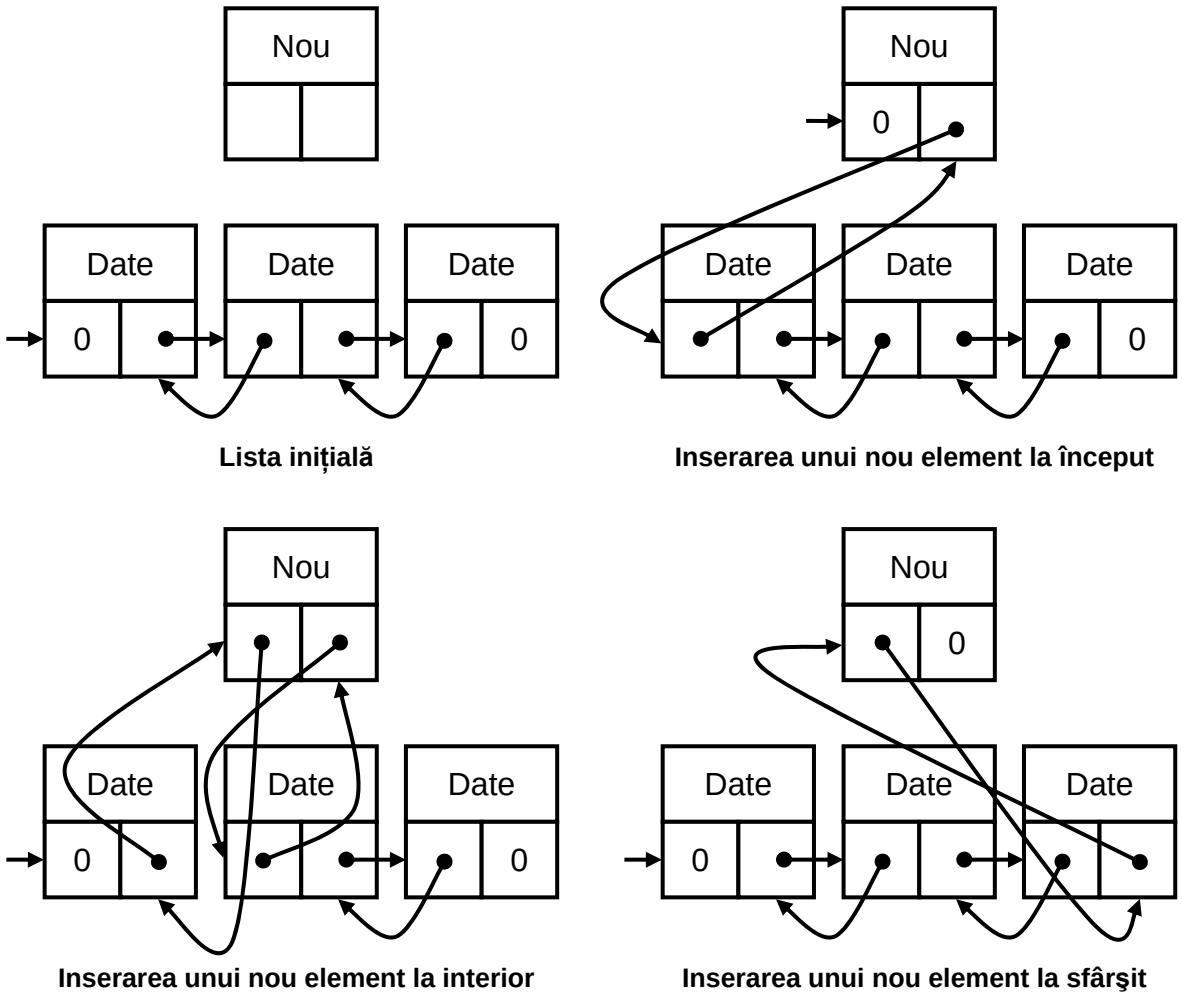


Figura 13.5 Inserarea unui nou element într-o listă dublu înlănțuită

Construirea unei liste dublu înlănțuite este un proces asemănător celui de alcătuire a unei liste simplu înlănțuite, cu deosebirea că sunt necesare două legături. Ca atare, structura trebuie să asigure spațiu pentru ambele. Utilizând din nou exemplul cu lista de adrese, structura adresă poate fi modificată după cum urmează, pentru a asigura ambele legături:

```
struct adresa {
 char nume[40];
 char strada[40];
 char oras[20];
 char stat[3] ;
 char cod_postal[11];
 struct adresa *urm;
 struct adresa *preced;
 }  info;
```

Utilizând *adresa* ca articol de date fundamental, funcția `store_ld()` alcătuieşte o listă dublu înlănțuită:

```
  void store_ld(struct adresa *i, struct adresa **ultim)
  {
   if(!*ultim) *ultim=i; /* Primul articol din lista*/
   else (*ultim)->urm = i;
   i->urm = NULL;
   i->preced = *ultim;
   *ultim = i;
  }
```

Funcția `store_ld()` inserează fiecare nouă intrare la sfârşitul listei. Funcția trebuie apelată cu un pointer către datele care trebuie stocate, precum şi cu un pointer către sfârşitul listei, care trebuie să fie nul la prima apelare.
Asemănător listelor simplu înlănțuite, o listă dublu înlănțuită poate fi construită prin intermediul unei funcții care stochează fiecare element într-o anumită locație din listă, în loc de a plasa fiecare element la sfârşitul listei. Funcția prezentată în continuare, `store_ld_sort()`, creează o listă sortată în ordine crescătoare:

```
/* Creează o lista ordonata dublu inlantuita. */
void store_ld_sort(
struct adresa *i, /* elementul nou */
struct adresa **prim, /* primul element pe lista */
struct adresa **ultim /* ultimul element pe lista */
)
{
 struct adresa *vechi, *p;
 if(*ultim==NULL) { /* primul element pe lista */
  i->urm = NULL;
  i->preced = NULL;
  *ultim = i;
  *prim = i;
  return;
 }
 p = *prim; /* incepe in varful listei */
 vechi = NULL; while(p) {
 if(strcmp(p->nume, i->nume)<0) {
  vechi = p;
  p = p->urm;
 }
 else {
  if(p->preced) {
   p->preced->urm = i;
   i->urm = p;
   i->preced = p->preced;
   p->preced = i;
   return;
  }
 i->urm = p; /* noul prim element */
 i->preced = NULL;
 p->preced = i;
 *prim = i;
 return;
 }
}
vechi->urm = i; /* inserează la sfârşit */
 i->urm = NULL;
 i->preced =* vechi;
 *ultim = i;
}
```

Deoarece primul sau ultimul element din listă se poate modifica, funcția `store_ld_sort()` actualizează în mod automat pointerii către elementele de început şi de sfârşit ale listei prin intermediul parametrilor prim şi ultim. Funcția trebuie apelată printr-un pointer către datele care urmează a fi stocate şi doi pointeri către primul, respectiv ultimul element din listă. Când funcția este apelată pentru prima dată, obiectele indicate de *prim* şi *ultim* trebuie să fie nule.

Ca şi în cazul listelor simplu înlănțuite, recuperarea unui anumit articol de date dintr-o listă dublu înlănțuită se rezumă la urmărirea legăturilor până la localizarea elementului corespunzător.
Există trei cazuri care trebuie luate în considerare la ştergerea unui element dintr-o listă dublu înlănțuită: ştergerea primului articol, ştergerea unui articol dintre două articole şi ştergerea ultimului articol. Fig 13.6 prezintă modul de rearanjare a legăturilor.

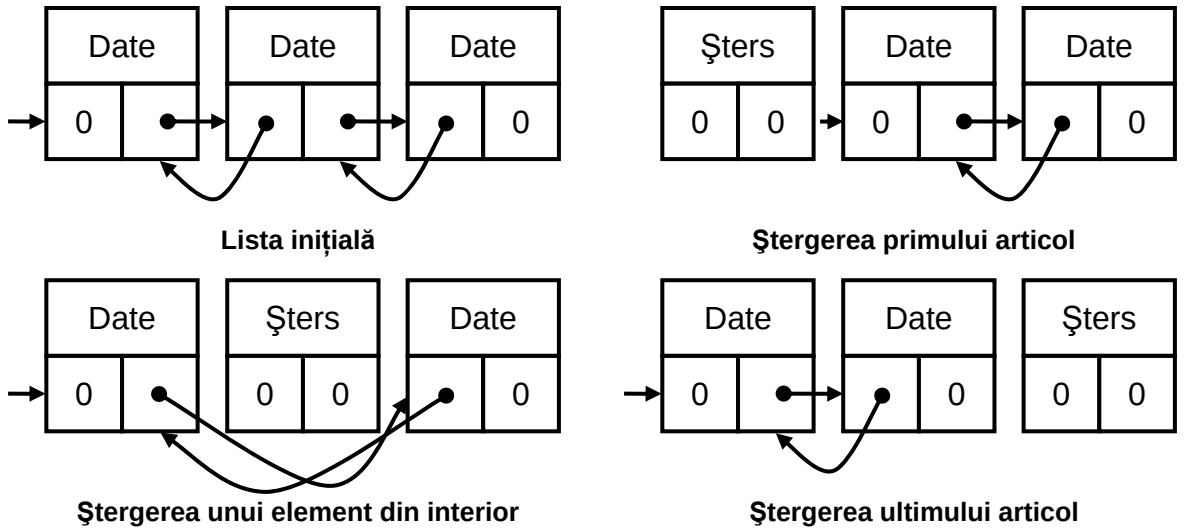


Figura 13.6 Ştergerea dintr-o listă dublu înlănțuită

Funcția `sterg_ld()`, prezentată mai jos, şterge un articol dintr-o listă dublu înlănțuită.

```
void sterg_ld (
struct adresa *a,  /* articolul care va fi şters */
struct adresa **prim,  /* primul articol */
struct adresa **ultim)  /* ultimul articol */
{
 if(a->preced) a->preced->urm = i->urm;
 else { /* noul prim element */
  *prim = a->urm;
  if(prim) prim->preced = NULL;
 }
 if(a->urm) a->urm->preced = a->preced;
 else /* şterge ultimul element */
 *ultim = a->preced;
}
```

Deoarece pot fi şterse şi primul sau ultimul element din listă, funcția `sterg_ld()` actualizează în mod automat pointerii către primul şi ultimul element din listă, prin intermediul parametrilor *prim* şi *ultim*. Funcția trebuie apelată printr-un pointer către datele care trebuie şterse, precum şi un pointer către pointerii primului şi ultimului articol din listă.

13.2 Cozi

O ***coadă*** (*queue*) este o listă simplu înlănțuită care este accesată în ordinea *first in, first out* - FIFO (primul care intră, primul care iese).

Cu alte cuvinte, primul articol din coadă este primul recuperat, al doilea de pe listă este al doilea recuperat etc. Acesta este singurul mod de stocare şi recuperare a datelor dintr-o coadă, nefiind permis accesul aleator al unui anumit articol.

Pentru a vizualiza modul de funcționare al unei cozi, să considerăm două funcții, `stochez()` şi `recuperez()`. Funcția `stochez()` aşează un articol la sfârşitul unei cozi, iar `recuperez()` elimină primul element din coadă şi returnează valoarea acestuia. Tabelul 13.1 ilustrează o serie de asemenea operații.

Tabelul 13.1 Operații cu cozi

Acțiunea	**Conținutul cozii**
`stochez(A)`	A
`stochez(B)`	A B
`stochez(C)`	A B C
`recuperez()` returnează A	B C
`stochez(D)`	B C D
`recuperez()` returnează B	C D
`recuperez()` returnează C	D

Nu uitați că o operație de recuperare elimină un articol din coadă şi îl distruge, dacă acesta nu este stocat în altă parte. Ca atare, este posibil ca o coadă să fie goală, deoarece toate componentele sale au fost înlăturate.

Pentru a vedea un exemplu de coadă în acțiune, vom folosi un program simplu de planificare a întâlnirilor.

Exemplul 13.1 Program simplu de planificare a întâlnirilor

Acest program permite introducerea unui număr de întâlniri, apoi, pe măsură ce întâlnirile au avut loc, acestea sunt şterse de pe listă. Din rațiuni de simplitate, fiecare descriere a unei întâlniri va fi limitată la 255 caractere, iar numărul acestora a fost arbitrar limitat la 100.

```
/* Mini-planificator de intalniri */
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>
#define MAX 100

char *p[MAX], *recuperez(void);
int poz_stoc = 0; int poz_rec = 0;
void    introduc(void),    stochez(char    *q),    listez(void),
          sterg(void);

void main(void)
{
 char s[80]; register int t;
 for(t=0; t<MAX; ++t) p[t] = NULL; /*init. tablou la 0 */
  for(;;) {
   printf("Introduc, Listez, Sterg, Terminare: ");
   gets(s);
   *s = toupper(*s);
   switch(*s) {
    case 'I': introduc(); break;
    case 'L': listez(); break;
    case 'S': sterg(); break;
    case 'T': exit(0);
   }
  }
}
```

```
/* introducerea intalnirilor în coada */
void introduc(void)
{
 char s[256], *p;
 do{
  printf("Introduceti intalnirea %d: ", poz_stoc+1);
  gets(s) ;
  if(*s==0) break; /* nu mai sunt locuri libere */
  p=(char *)malloc(strlen(s)+1);
  if(!p) {
   printf("Memorie epuizata.\n"); return; }
  strcpy(p,s);
  if(*s) stochez(p);
 }
 while (*s);
}
/* Consulta coada. */
void listez(void)
{
 register int t;
 for(t = poz_rec; t<poz_stoc; ++t)
  printf("%d. %s\n", t+1, p[t]);
}
/* Sterge o intalnire din coada. */
void sterg(void)
{
 char *p;
 if((p=recuperez())==NULL) return; printf("%s\n", p);
}
/* Stocheaza o intalnire. */
void stochez(char *q)
{
 if(poz_stoc==MAX) {
  printf("Lista completa\n" ) ;
  return;
 }
 p[poz_stoc] = q; poz_stoc++;
}
/* Recupereaza o intalnire. */
char *recuperez(void)
{
 if(poz_rec==poz_stoc) {
  printf("Nu mai sunt intalniri.\n"); return NULL; }
 poz_rec++;
 return p[poz_rec-1];
}
```

Funcțiile `stochez()` şi `recuperez()` din program vor stoca pointeri către şirurile care descriu întâlnirile.

Observați că aceste funcții necesită două variabile globale: `poz_stoc` (care memorează indicele următoarei locații libere de memorare) şi `poz_rec` (care stochează indicele următorului articol care va fi recuperat). Aceste funcții pot fi folosite pentru a menține o coadă de alte tipuri de date prin simpla modificare a tipului de bază al tabloului cu care operează acestea.

Funcția `stochez()` amplasează pointerii noilor întâlniri către sfârşitul listei şi verifică dacă lista este completă. Funcția `recuperez()` preia întâlnirile din coadă în timpul desfăşurării unor evenimente. La fiecare nouă întâlnire programată, `poz_stoc` este incrementată şi, la fiecare întâlnire desfăşurată, `poz_rec` este incrementată. În esență, `poz_rec` este mereu pe urmele lui `poz_stoc` pe toată lungimea cozii. Dacă `poz_rec` şi `poz_stoc` sunt egale, nu mai există evenimente de programat. Chiar dacă informația stocată într-o coadă nu este practic distrusă de `recuperez()`, este totuşi distrusă fiindcă nu mai poate fi accesată niciodată.

13.2.1 Cozi circulare

E posibil ca, în timpul studiului programului precedent, să vă fi gândit deja la o îmbunătățire. De exemplu, în loc de a cere programului să se oprească la limita tabloului folosit pentru stocarea cozii, puteți cere indicelui de stocare (`poz_stoc`) şi a celui de recuperare (`poz_rec`) să salte înapoi la începutul tabloului. Astfel, în coadă se pot plasa oricâte articole, atâta timp cât de asemenea se extrag articole din tablou. Această implementare a unei cozi se numeşte *implementare circulară,* deoarece îşi foloseşte tabloul de stocare ca şi cum acesta ar fi un cerc, şi nu o listă liniară.
Pentru a crea o coadă circulară în programul de planificare, este necesar să modificați funcțiile `stochez()` şi `recuperez()` după cum urmează:

```
void stochez(char *q) {
/* Coada e plina daca poz_stoc este cu o unitate mai mic decât poz_rec sau daca
            poz_stoc este la sfarsitul tabloului care stochează coada si poz_rec este la
            inceputul acestuia. */
if(poz_stoc+1==poz_rec || (poz_stoc+1)==MAX && !poz_rec)) {
 printf("Lista completa\n"); return;
 p[poz_stoc] = q;
 poz_stoc++;
 if(poz_stoc==MAX) poz_stoc = 0; /* salt inapoi */
}

char *recuperez(void)
{
 if(poz_rec==MAX) poz_rec = 0; /* salt inapoi */
 if(poz_rec==poz_stoc) {
  printf("Nu sunt evenimente in lista.\n");
  return NULL;
 }
 poz_rec++;
 return p[poz_rec-1];
}
```

În esență, coada este completă când indicele de stocare este cu o unitate mai mic decât indicele de recuperare; în caz contrar, în coadă există spațiu pentru un alt eveniment. Conceptual, tabloul folosit pentru programul de planificare se prezintă ca în Fig 13.7.

Poate cea mai comună utilizare a unei cozi circulare se regăseşte în sistemele de operare, acolo unde o coadă circulară memorează informația citită din şi scrisă pe fişiere, pe disc sau în consolă. Cozile circulare sunt de asemenea folosite în programe de aplicație în timp real, care trebuie să continue să proceseze informația în timpul memorării temporare a

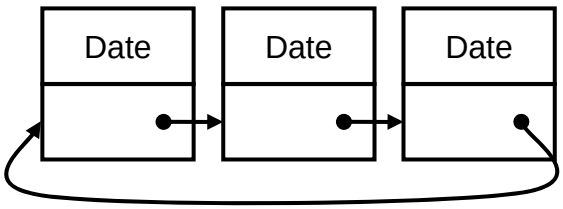


Figura 13.7 Coadă circulară

cererilor de I/O. Multe procesoare de texte procedează astfel când reformatează un paragraf sau aliniază o linie. Informația tastată nu este afişată până la încheierea operației. Pentru a realiza aceasta, programul de aplicație trebuie să controleze intrările de la tastatură în timpul execuției celuilalt proces. Dacă se apasă vreo tastă, aceasta este rapid introdusă în coadă şi execuția celuilalt proces continuă. După încheierea procesului, caracterele sunt reproduse din coadă.

13.3 Stive

O ***stivă*** *(stack)* este o listă simplu înlănțuită care este accesată în ordinea *last in, first out* - LIFO (ultimul care intră, primul care iese).

Pentru a vă imagina o stivă, gândiți-vă la un vraf de farfurii. Prima farfurie de pe masă este ultima care va fi folosită, iar ultima farfurie aşezată în stivă este şi prima care va fi folosită. Stivele sunt foarte des folosite în programele de sistem, inclusiv în compilatoare şi interpretoare. De fapt, compilatoarele de C folosesc stivele pentru a transfera argumente către funcții. Când se lucrează cu stive, cele două operații de bază - anume memorarea şi citirea - sunt numite în mod tradițional *push* (a împinge) şi *pop* (a scoate). Ca atare, pentru a implementa o stivă este nevoie de două funcții, şi anume **push(),** care amplasează o valoare în stivă, şi **pop(),** care citeşte o valoare din stivă. De asemenea, este necesară utilizarea unei regiuni de memorie drept stivă. În acest scop se poate folosi un tablou sau se poate aloca o regiune de memorie folosind funcțiile de alocare dinamică ale limbajului C. Ca şi în cazul cozilor, funcția de citire preia o valoare din listă şi o distruge, dacă aceasta nu este stocată în altă parte. În continuare sunt prezentate formele generale ale unor funcții **push()** şi **pop()** care folosesc un tablou de întregi. Se pot păstra stivele de alte tipuri de date modificând tipul de bază al tabloului cu care operează funcțiile **push()** şi **pop().**

```
int stack[MAX];
int vfs = 0; /* vârful stivei */
/* Introducerea unui element in stiva. */
void push(int i)
{
if(vfs>=MAX) {
 printf ( "Stiva plina"); return; }
stack[vfs] = i; vf s++;
}

/* Citeşte elementul din vârful stivei. */
pop(void)
{
 vfs--;
 if(vfs<0) {
  printf("Depăşire a stivei in sens negativ\n"); return 0;}
 return stack[vfs];
}
```

Variabila **vfs** este indicele vârfului stivei. La implementarea acestor funcții, nu trebuie să uitați de prevenirea depăşirilor în sens pozitiv sau negativ ale stivei. În aceste rutine, o stivă goală este semnalată printr-un **vfs** egal cu zero, iar o stivă **plină** este indicată printr-un **vfs** superior ultimei locații de stocare. Pentru a vedea cum funcționează o stivă, vezi Tabelul 13.2.

Tabelul 13.2 Operații cu stive

Acțiune	Conținutul stivei
push(A)	A
push(B)	B A
push(C)	C B A
pop() citeşte C	B A
push(F)	F B A
pop() citeşte F	B A
pop() citeşte B	A
pop() citeşte A	stivă goală

Un excelent exemplu de utilizare a unei stive o reprezintă un calculator cu patru funcții. Majoritatea calculatoarelor acceptă în prezent o formă standard a unei expresii denumite *notație infix,* care ia forma generală *operand-operator-operad.* De exemplu, pentru a aduna 100 cu 200, mai întâi se introduce 100, apoi se apasă tasta PLUS (+), apoi se introduce 200, şi în fine se apasă tasta EGAL (=). Spre deosebire de acestea, multe calculatoare mai vechi (din care unele se mai produc şi în prezent), folosesc *notația postfix,* în cadrul căreia se introduc mai întâi operanzii şi apoi operatorul. De exemplu, pentru a aduna 100 cu 200 folosind notația postfix, mai întâi se introduce 100, apoi 200, iar apoi se apasă tasta PLUS. Pe măsură ce operanzii sunt introduşi, aceştia sunt aşezați într-o stivă. De fiecare dată când este introdus un operator, cei doi operanzi sunt eliminați din stivă, iar rezultatul este introdus în stivă. Un avantaj al notației postfix este acela că utilizatorul poate introduce expresii lungi, complexe.

Exemplul 13.2 Calculator postfix pentru expresii întregi

Exemplul următor demonstrează modul de utilizare al unei stive prin implementarea unui calculator postfix pentru expresii întregi.

```
/* Un calculator simplu cu patru functii. */
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int *p;  /* pointer către o regiune de memorie libera */
int *vfs;  /* pointer către vârful stivei */
int *bzs;  /* pointer către baza stivei */

void push(int i); int pop(void); void main(void)
{
int a, b; char s[80];
p = (int *) malloc(MAX*sizeof(int));  /* memorie stiva */
if(!p) {
 printf("Eroare de alocare\n"); exit(1); )
vfs = p; bzs = p+MAX-1;
printf("Calculator cu patru functii\n");
printf("Tastați 'q' pentru a renunta\n");
do {
 printf(": "); gets (s) ;
 switch(*s) {
  case '+':
          a = pop(); b = pop();
          printf("%d\n", a+b); push(a+b); break;
  case '-':
          a = pop(); b = pop();
          printf('"td\n", b-a); push(b-a); break;
  case '*':
```

```
            a = pop(); b = pop();
            printf("*d\n", b*a); push(b*a) ; break;
    case '/':
            a = pop(); b = pop();
            if(a==0) { printf("impartire cu zero\n"); break;}
            printf("%d\n", b/a); push(b/a); break;
    case '.': /* arata vârful stivei */
            a = pop(); push(a);
            printf("valoarea in varf: %d\n", a); break;
    default: push(atoi(s));
   }
   }  while(*s!='q');
  }
  /* introduce un element in stiva */
  void push(int i)
  {
   if(p>bzs) {
    printf("Stiva plina\n"); return;}
   *p = i; p++;
  }
  /* Citeşte elementul din vârful stivei. */
  pop(void)
  {
   p--;
   if(p<vfs) { printf("Depăşire in sens negativ\n"); return 0; }
  return *p;
  }
```

Aşa cum se observă, a fost necesară modificarea funcțiilor **push()** şi **pop()**, funcții care au fost folosite pentru memorarea stivei într-o zonă de memorie alocată dinamic, în loc de un tablou de dimensiune fixă. Deşi utilizarea memoriei dinamic alocate nu era necesară pentru acest exemplu simplu, este prezentat modul de utilizare a memoriei dinamic alocate în lucrul cu stive. Înainte de a putea folosi aceste funcții, a fost necesară alocarea unei zone libere de memorie folosind `malloc()`, adresa începutului acelei regiuni trebuind alocată variabilei `vfs`, iar adresa sfârşitului regiunii trebuind alocată regiunii `bzs`.

13.4 Arbori binari

Ultima structură de date examinată o constituie *arborele binar (binary tree).* Deşi există mai multe categorii de arbori, cei binari sunt o categorie specială deoarece, atunci când sunt sortați, se pretează la căutări, inserții şi ştergeri rapide. Fiecare articol dintr-un arbore este alcătuit din informații şi legături cu membrul din stânga şi cu cel din dreapta. **Error! Reference source not found.** prezintă un arbore de mici dimensiuni.

Pentru discuțiile referitoare la arbori este necesară o terminologie specială. Specialiştii în calculatoare nu sunt vestiți pentru gramatica lor, ca atare terminologia aferentă arborilor reprezintă o veritabilă metaforă a confuziei. Primul articol al unui arbore îl constituie *rădăcina (root).* Fiecare articol de date este denumit *nod (node),* iar orice componentă a arborelui este cunoscută sub numele de *subarbore (subtree).* Un nod fără *subarbori* este denumit *nod terminal (terminal node)* sau *frunză (leaf). Înălțimea* unui arbore este egală cu numărul de nivele pe care se extinde rădăcina acestora. Când lucrați cu arbori, vă puteți imagina dispunerea lor în memorie exact ca şi pe hârtie. Nu uitați însă că un arbore reprezintă numai o modalitate de organizare logică a datelor în memorie, iar memoria este liniară.

Într-un fel, arborele binar este o formă specială de listă înlănțuită. Articolele pot fi accesate, inserate sau şterse în orice ordine. De asemenea, operația de recuperare nu este distructivă. Deşi arborii sunt uşor de vizualizat, prezintă totuşi probleme de programare foarte dificile. Această discuție nu reprezintă decât un timid început.
Majoritatea funcțiilor care folosesc arbori sunt recursive, deoarece arborele însuşi este o structură de date recursivă. Cu alte cuvinte, fiecare subarbore este el însuşi un arbore. Ca atare, rutinele despre care vom discuta aici vor fi tot recursive. Nu uitați că există şi versiuni nerecursive ale acestor funcții, dar sunt mult mai dificil de înțeles.
Modul de organizare a unui arbore depinde de modul în care va fi referit acesta. Procesul de accesare a fiecărui nod dintr-un arbore se numeşte *traversarea arborelui (tree traversal).* Fie arborele din figura Figura 13.8.
Există trei modalități de traversare a unui arbore: *inordine, preordine* şi *postordine.* Dacă se foloseşte inordinea, se parcurge subarborele stâng, apoi rădăcina şi apoi subarborele drept. Modul preordine implică succesiunea rădăcină, subarbore stâng, subarbore drept, iar în modul postordine se pleacă din subarborele stâng, subarborele drept şi se ajunge în rădăcină. Ordinea de acces în cele trei metode este

- inordine a b c d e f g
- preordine d b a c f e g
- postordine a c b e g f d

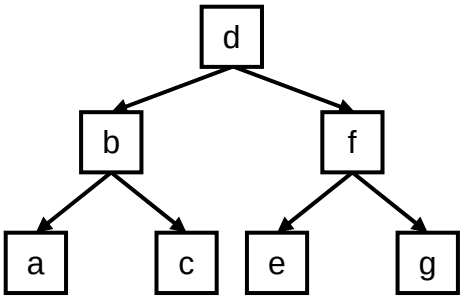


Figura 13.8 Arbore simplu

Deşi un arbore nu trebuie sortat, în majoritatea cazurilor această operație este necesară. Desigur, ceea ce alcătuieşte un arbore sortat depinde de modul în care va fi traversat arborele. În continuare, vom presupune folosit modul inordine. În consecință, un arbore binar sortat este acela în care subarborele stâng conține noduri mai puține sau egale cu rădăcina, iar nodurile de pe dreapta sunt mai mari decât rădăcina.
Exemplul 13.3 Creare şi afişare arbore
Să luăm în considerare un program mic, dar interesant, care alcătuieşte un arbore binar sortat, iar apoi afişează arborele în mod inordine, pe marginile ecranului.
În continuare prezentăm integral programul de afişare a arborelui. Încercați diferiți arbori pentru a testa modul de alcătuire al fiecăruia.

```
#include <stdlib.h>
#include <stdio.h>

struct arbore {
 char info;
 struct arbore *stang;
 struct arbore *drept;
};

struct arbore *radacina; /* primul nod din arbore */
struct arbore *sort_arb(struct arbore *radacina, struct arbore
          *r, char info);
void print_arbore(struct arbore *radacina, int l) ;

void main(void)
{
 char s[80];
 radacina = NULL; /* initializarea radacinii */
 do{
```

```
    printf("introduceti o litera: ");
    gets(s);
    if(!radacina) radacina=sort_arb(radacina, radacina, *s);
    else sort_arb(radacina,radacina,*s); }
   while(*s);
   print_arbore(radacina, NULL);
  }

  struct arbore *sort_arb(struct arbore *radacina, struct arbore
            *r, char info)
  {
   if(!r) {
    r = (struct arbore *) malloc(sizeof(struct arbore));
    if(!r) {printf("Memorie epuizata\n"); exit(0);}
    r->stang = NULL;
    r->drept = NULL;
    r->info = info;
    if(!radacina) return r; /* prima intrare */
    if(info<radacina->info) radacina->stang = r;
    else radacina->drept = r;
    return r;
   }
   if(info<r->info) sort_arb(r, r->stang, info);
   else sort_arb(r, r->drept, info);
  }

  void print_arbore(struct arbore *r, int l)
  {
   int i;
   if(r==NULL) return;
   print_arbore(r->drept, l+1);
   for(i=0; i<l; ++i)
    printf(" ");
   printf("%c\n", r->info);
   print_arbore(r->stang, l+1);
  }
```

Funcția `sort_arb()` construieşte un arbore binar sortat. Algoritmul funcției nu face decât să urmeze legăturile spre stânga sau spre dreapta în funcție de câmpul `info`. Pentru a utiliza această funcție, este necesară o variabilă globală care indică spre rădăcina arborelui. Această globală trebuie inițial setată la NULL. Valoarea returnată a primului apel la `sort_arb()` trebuie atribuită acestui pointer. La următoarele apeluri nu mai este necesară reatribuirea rădăcinii. Presupunând că numele globalei este `rad`, pentru apelarea funcției `sort_arb()` se foloseşte secvența:

```
/* apelarea funcției sort_arb() */
if(!rad) rad = sort_arb(rad, rad, info);
else sort_arb(rad, rad, info);
```

Astfel, atât primul element, cât şi cele ulterioare acestuia, pot fi inserate corect.
Funcția `sort_arb()` este un algoritm recursiv, ca majoritatea rutinelor care operează cu arbori. Dacă se folosesc metodele iterative, aceeaşi rutină devine de câteva ori mai lungă. Funcția trebuie apelată folosind următoarele argumente (de la stânga la dreapta): un pointer către rădăcina întregului arbore, un pointer către rădăcina următorului arbore în care se va căuta şi informația care va fi memorată. La prima apelare a funcției, primii doi parametri reprezintă, ambii, pointeri către rădăcina întregului arbore. Din motive de

claritate, ca informație stocată în arbore se va considera un singur caracter. Totuşi, programatorul poate utiliza orice tip de date.
Pentru a traversa în **ordine** arborele alcătuit folosind funcția `sort_arb()` şi pentru a afişa câmpul `info` al fiecărui nod, se poate folosi funcția `in_ordine()`, prezentată în continuare:

```
void in_ordine(struct arbore *radacina)
{
 if(!radacina) return;
 in_ordine(radacina->stanga);
 if(radacina->info) printf("%c ", radacina->info);
 in_ordine(radacina->dreapta);
}
```

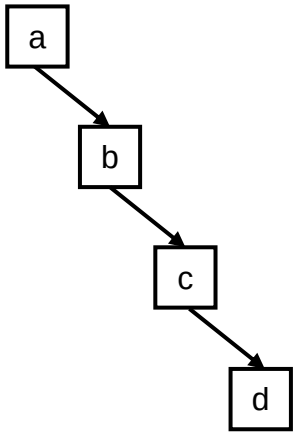
Această funcție recursivă returnează în momentul întâlnirii unui nod terminal (adică un pointer nul).
Funcțiile pentru traversarea arborelui în modurile **preordine** şi **postordine** sunt prezentate în continuare.

```
void pre_ordine(struct arbore *radacina)
{
 if(!radacina) return;
 if(radacina->info) printf("%c ", radacina->info);
 pre_ordine(radacina->left);
 pre_ordine(radacina->right);
}

void post_ordine(struct arbore *radacina)
{
 if(!radacina) return;
 post_ordine(radacina->left) ; post_ordine(radacina->right);
 if(radacina->info) printf("%c ", radacina->info);
}
```

Pentru a afişa arborele, programul necesită o versiune uşor modificată a funcției `in_ordine()`. Deoarece arborele este afişat pe marginile ecranului, subarborele drept trebuie afişat înaintea celui stâng, pentru ca arborele să arate corect. (Aceasta reprezintă, din punct de vedere tehnic, opusul unei traversări în mod inordine.) Această funcție nouă este funcția `print_arbore()` din program.

Programul sortează efectiv informația furnizată şi reprezintă în esență, o variație a sortării pe care ați urmărit-o în capitolul precedent. În caz general, performanțele acestui program sunt bune, dar sortarea rapidă rămâne o metodă superioară de sortare de uz general, deoarece necesită mai puțină memorie şi mai puține operații de procesare. Totuşi, dacă trebuie să alcătuiți un arbore pornind de la zero sau dacă trebuie să întrețineți un arbore deja sortat, trebuie să introduceți noile intrări în ordinea de sortare, folosind funcția `sort_arb()`.



Dacă ați rulat programul de afişare a arborelui, ați constatat probabil că unii arbori sunt *în echilibru* - altfel spus, fiecare subarbore are aproximativ aceeaşi înălțime cu a celorlalți, iar restul sunt foarte neechilibrați. De fapt, dacă ați inserat arborele *abcd***,** acesta va arăta ca în figura alăturată, deci nu mai există subarbori. Acesta se numeşte *arbore degenerat,* deoarece s-a transformat într-o listă liniară. În general, dacă datele de intrare folosite la alcătuirea unui arbore binar au o distribuție oarecum aleatoare, arborele generat va aproxima un arbore echilibrat. (Este posibilă reajustarea arborelui la

fiecare inserție de date pentru menținerea arborelui în echilibru, dar este un proces complex care depăşeşte cadrul acestui capitol.)
Funcțiile de căutare sunt uşor de implementat pentru arborii binari. Următoarea funcție returnează un pointer către nodul care este identic cu cheia; în caz contrar, returnează zero.

```
struct arbore *search_arbore(struct arbore, *radacina, char
          cheia)
{
 if(!radacina) return radacina; /* arbore gol */
 while(radacina->info!=cheia) {
  if(cheia<radacina->info) radacina = radacina->stang;
  else radacina = radacina->drept;
  if(radacina==NULL) break;
 }
 return radacina;
}
```

Din păcate, ştergerea unui nod dintr-un arbore nu este atât de simplă ca o operație de căutare. Nodul şters poate fi rădăcina, un nod din stânga sau un nod din dreapta. De asemenea, este posibil ca nodul să aibă între zero şi doi subarbori ataşați acestuia. Procesul de rearanjare a pointerilor se pretează la un algoritm recursiv, care este prezentat în cele ce urmează:

```
struct arbore *sterg_arb(struct arbore *radacina, char cheia)
{
 struct arbore *p, *p2;
 if(!radacina) return radacina; /* nu a fost găsit */
 if(radacina->info==cheia) { /* sterge rădăcina */
/* adică arborele este gol */
 if(radacina->stang==radacina->drept)
  { free(radacina); return NULL;}
/* sau daca un subarbore este nul */
 else if(radacina->stang ==NULL) {
   p = radacina->drept;
   free(radacina);
   return p;
  }
 else if(radacina->drept==NULL) {
   p = radacina->stang;
   free(radacina);
   return p;
 }
/* sau ambii subarbori exista */
else {
  p2 = radacina->drept;
  p = radacina->drept;
  while(p->stang) p = p->stang;
  p->stang = radacina->stang;
  free(radacina);
  return p2;
 }
}
 if(radacina->info->cheia)          radacina->drept          ==
          sterg_arb(radacina->drept,cheia);
 else radacina->stang = sterg_arb(radacina->stang, cheia);
```

```
  return radacina;
}
```

Nu uitați să actualizați pointerul rădăcinii în restul programului, fiindcă nodul şters ar putea fi rădăcina arborelui. Cel mai simplu mod de a realiza aceasta este de a atribui valoarea returnată de `sterg_arb()` variabilei din programul dvs. care indică spre rădăcină, utilizând o apelare similară celei ce urmează:

```
radacina = sterg_arb(radacina,cheia);
```

Arborii binari oferă o deosebită putere, flexibilitate şi eficiență când sunt folosiți cu programe de administrare a bazelor de date, deoarece informațiile aferente acestor baze de date trebuie să se afle pe disc, iar timpii de acces sunt importanți. Deoarece un arbore binar în echilibru are, în cel mai rău caz, de efectuat $\log_2 n$ comparații, este mult mai convenabil de utilizat decât o listă înlănțuită, care implică o căutare secvențială.